

LN14. 核方法

何 琨

计算机学院数据挖掘与机器学习实验室

华中科技大学 Hopcroft 计算科学研究中心

邮箱: brooklet60@hust.edu.cn

2026 年 4 月



- ① 特征扩展
 - 手动特征扩展
 - 手动特征扩展
 - 手动特征扩展
- ② 核技巧
 - 核技巧
 - 均方损失梯度下降
 - 内积计算
- ③ 通用的核方法
 - 常见的核函数
 - 核函数

- 1 特征扩展
 - 手动特征扩展
 - 手动特征扩展
 - 手动特征扩展
- 2 核技巧
 - 核技巧
 - 均方损失梯度下降
 - 内积计算
- 3 通用的核方法
 - 常见的核函数
 - 核函数

动机

- 线性分类器的性能很好，但如果不存在线性的决策边界呢？
- 有一种巧妙的方法可以将非线性纳入大多数线性分类器。

手动特征扩展

- 可以通过在输入特征向量上应用基函数 (特征变换)，来使线性分类器变为非线性分类器。
- 问题：设有两类数据，位于二维平面的两个不同半径的中心圆附近，请问是否能进行线性分类？

如何对非线性的数据进行线性分类？

- 问题：设有两类数据，位于二维平面的两个不同半径的中心圆附近，请问是否能进行线性分类？

小故事：阿凡提巧分粟与沙

- 一天，有位农夫焦急地找到阿凡提，说：“阿凡提，我的粮仓里不小心混进了很多沙子，现在粟米和沙子混在一起，怎么才能分开呢？”



小故事：阿凡提巧分粟与沙

- 一天，有位农夫焦急地找到阿凡提，说：“阿凡提，我的粮仓里不小心混进了很多沙子，现在粟米和沙子混在一起，怎么才能分开呢？”
- 阿凡提想了一下，笑着说：“这很简单，拿来一些水，我来帮你。”
- 农夫按照阿凡提的吩咐取来了水，阿凡提把一部分混合物倒进水里。只见粟米浮在水面，而沙子则沉到了底部。阿凡提用筛子轻轻一捞，粟米就被分离出来了。之后等待晒干就能继续使用了。

在中国古代，淘金者会用水流分离金子和泥沙。因为黄金比泥沙重，在水中会沉到底，而泥沙会随水流冲走。这种方法被广泛用于金矿开采。



小故事：阿凡提巧分粟与沙

- 一天，有位农夫焦急地找到阿凡提，说：“阿凡提，我的粮仓里不小心混进了很多沙子，现在粟米和沙子混在一起，怎么才能分开呢？”
- 阿凡提想了一下，笑着说：“这很简单，拿来一些水，我来帮你。”
- 农夫按照阿凡提的吩咐取来了水，阿凡提把一部分混合物倒进水里。只见粟米浮在水面，而沙子则沉到了底部。阿凡提用筛子轻轻一捞，粟米就被分离出来了。之后等待晒干就能继续使用了。

在中国古代，淘金者会用水流分离金子和泥沙。因为黄金比泥沙重，在水中会沉到底，而泥沙会随水流冲走。这种方法被广泛用于金矿开采。



小故事：阿凡提巧分粟与沙

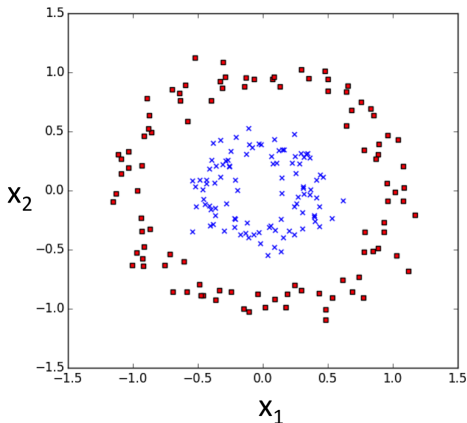
- 一天，有位农夫焦急地找到阿凡提，说：“阿凡提，我的粮仓里不小心混进了很多沙子，现在粟米和沙子混在一起，怎么才能分开呢？”
- 阿凡提想了一下，笑着说：“这很简单，拿来一些水，我来帮你。”
- 农夫按照阿凡提的吩咐取来了水，阿凡提把一部分混合物倒进水里。只见粟米浮在水面，而沙子则沉到了底部。阿凡提用筛子轻轻一捞，粟米就被分离出来了。之后等待晒干就能继续使用了。

在中国古代，淘金者会用水流分离金子和泥沙。因为黄金比泥沙重，在水中会沉到底，而泥沙会随水流冲走。这种方法被广泛用于金矿开采。



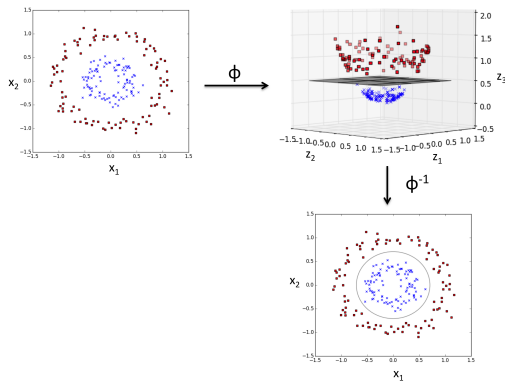
如何对非线性的数据进行线性分类？

- 问题：设有两类数据，位于二维平面的两个不同半径的中心圆附近，请问是否能进行线性分类？



如何对非线性的数据进行线性分类？

- 问题：设有两类数据，位于二维平面的两个不同半径的中心圆附近，请问是否能进行线性分类？
- 可通过在输入特征向量上应用基函数 (特征变换)，使线性分类器变为非线性分类器。
- 形式上，对于数据向量 $\mathbf{x} \in \mathbb{R}^d$ ，应用转换 $\mathbf{x} \rightarrow \phi(\mathbf{x})$ ，其中 $\phi(\mathbf{x}) \in \mathbb{R}^D$ 。
- 通常 $D \gg d$ ，因为添加了捕捉原始特征之间的非线性相互作用的维度。



优点

转化以后的问题保持凸性和良好的表现。

(也就是说, 仍然可以使用原来的梯度下降代码, 只是使用了更高维的表征)

缺点

$\phi(\mathbf{x})$ 的维度可能非常高。

手动特征扩展的示例

$$\text{设 } \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{pmatrix}, \text{ 并定义 } \phi(\mathbf{x}) = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_d \\ x_1 x_2 \\ \vdots \\ x_{d-1} x_d \\ \vdots \\ x_1 x_2 \cdots x_d \end{pmatrix}$$

测试

$\phi(\mathbf{x})$ 的维数是多少?

手动特征扩展的示例

- 设 $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{pmatrix}$, 并定义 $\phi(\mathbf{x}) = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_d \\ x_1 x_2 \\ \vdots \\ x_{d-1} x_d \\ \vdots \\ x_1 x_2 \cdots x_d \end{pmatrix}$

- 问: $\phi(\mathbf{x})$ 的维数是多少?
- 答: $\phi(\mathbf{x})$ 的维度为 2^d 。
- 这个新的表征 $\phi(\mathbf{x})$ 具有非常强的表征能力, 并且允许使用复杂的非线性决策边界。
- 但是它的维数非常高, 会导致算法运行非常慢。

- ① 特征扩展
 - 手动特征扩展
 - 手动特征扩展
 - 手动特征扩展
- ② 核技巧
 - 核技巧
 - 均方损失梯度下降
 - 内积计算
- ③ 通用的核方法
 - 常见的核函数
 - 核函数

核技巧是一种可以解决如上两难问题的方法。它学习高维空间中的函数，而不需要计算单个向量 $\phi(\mathbf{x})$ 或计算完整向量 $\mathbf{w}$ 。

观察

如果对任一标准损失函数使用梯度下降法，则**梯度是输入样本的线性组合**。

回顾：均方损失梯度下降

观察

如果对任一标准损失函数使用梯度下降法，则**梯度是输入样本的线性组合**。

例如，对于均方损失：

$$\ell(\mathbf{w}) = \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2$$

梯度下降规则：步长/学习率 $s > 0$ (前几讲中将之记为 $\alpha > 0$)，随着迭代的过程更新 $\mathbf{w}$

$$w_{t+1} \leftarrow w_t - s \left(\frac{\partial \ell}{\partial \mathbf{w}} \right) \quad \text{其中: } \frac{\partial \ell}{\partial \mathbf{w}} = \sum_{i=1}^n \underbrace{2(\mathbf{w}^\top \mathbf{x}_i - y_i)}_{\gamma_i : \text{function of } \mathbf{x}_i, y_i} \quad \mathbf{x}_i = \sum_{i=1}^n \gamma_i \mathbf{x}_i$$

可以将 $\mathbf{w}$ 表示为所有输入向量的线性组合，

$$\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{x}_i.$$

引理: $\mathbf{w}$ 是所有输入向量的线性组合。

$$\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{x}_i.$$

由于损失函数是凸的，最终的解决方案与初始值无关，因此可以初始化 $\mathbf{w}_0$ 为任意值。

为了方便起见，选择 $\mathbf{w}_0 = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$ 。

对于 $\mathbf{w}_0$ 的初始选择， $\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{x}_i$ 的线性组合很简单：

$$\alpha_1 = \cdots = \alpha_n = 0$$

均方损失梯度下降

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - s \left(\frac{\partial \ell}{\partial \mathbf{w}} \right) \quad \text{其中: } \frac{\partial \ell}{\partial \mathbf{w}} = \sum_{i=1}^n \underbrace{2(\mathbf{w}^\top \mathbf{x}_i - y_i)}_{\gamma_i} \mathbf{x}_i = \sum_{i=1}^n \gamma_i \mathbf{x}_i$$

要证明: $\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{x}_i$

在整个梯度下降的优化过程中, 这样的系数 $\alpha_1, \dots, \alpha_n$ 必须始终存在, 我们可以通过更新 α_i 的系数来重写梯度的更新:

$$\mathbf{w}_1 = \mathbf{w}_0 - s \sum_{i=1}^n 2(\mathbf{w}_0^\top \mathbf{x}_i - y_i) \mathbf{x}_i = \sum_{i=1}^n \alpha_i^0 \mathbf{x}_i - s \sum_{i=1}^n \gamma_i^0 \mathbf{x}_i = \sum_{i=1}^n \alpha_i^1 \mathbf{x}_i \quad (\text{with } \alpha_i^1 = \alpha_i^0 - s\gamma_i^0)$$

$$\mathbf{w}_2 = \mathbf{w}_1 - s \sum_{i=1}^n 2(\mathbf{w}_1^\top \mathbf{x}_i - y_i) \mathbf{x}_i = \sum_{i=1}^n \alpha_i^1 \mathbf{x}_i - s \sum_{i=1}^n \gamma_i^1 \mathbf{x}_i = \sum_{i=1}^n \alpha_i^2 \mathbf{x}_i \quad (\text{with } \alpha_i^2 = \alpha_i^1 - s\gamma_i^1)$$

$$\mathbf{w}_3 = \mathbf{w}_2 - s \sum_{i=1}^n 2(\mathbf{w}_2^\top \mathbf{x}_i - y_i) \mathbf{x}_i = \sum_{i=1}^n \alpha_i^2 \mathbf{x}_i - s \sum_{i=1}^n \gamma_i^2 \mathbf{x}_i = \sum_{i=1}^n \alpha_i^3 \mathbf{x}_i \quad (\text{with } \alpha_i^3 = \alpha_i^2 - s\gamma_i^2)$$

...

$$\mathbf{w}_t = \mathbf{w}_{t-1} - s \sum_{i=1}^n 2(\mathbf{w}_{t-1}^\top \mathbf{x}_i - y_i) \mathbf{x}_i = \sum_{i=1}^n \alpha_i^{t-1} \mathbf{x}_i - s \sum_{i=1}^n \gamma_i^{t-1} \mathbf{x}_i = \sum_{i=1}^n \alpha_i^t \mathbf{x}_i \quad (\text{with } \alpha_i^t = \alpha_i^{t-1} - s\gamma_i^{t-1})$$

均方损失梯度下降

$$\mathbf{w}_1 = \mathbf{w}_0 - s \sum_{i=1}^n 2(\mathbf{w}_0^\top \mathbf{x}_i - y_i) \mathbf{x}_i = \sum_{i=1}^n \alpha_i^0 \mathbf{x}_i - s \sum_{i=1}^n \gamma_i^0 \mathbf{x}_i = \sum_{i=1}^n \alpha_i^1 \mathbf{x}_i \quad (\text{with } \alpha_i^1 = \alpha_i^0 - s\gamma_i^0)$$

$$\mathbf{w}_2 = \mathbf{w}_1 - s \sum_{i=1}^n 2(\mathbf{w}_1^\top \mathbf{x}_i - y_i) \mathbf{x}_i = \sum_{i=1}^n \alpha_i^1 \mathbf{x}_i - s \sum_{i=1}^n \gamma_i^1 \mathbf{x}_i = \sum_{i=1}^n \alpha_i^2 \mathbf{x}_i \quad (\text{with } \alpha_i^2 = \alpha_i^1 - s\gamma_i^1)$$

$$\mathbf{w}_3 = \mathbf{w}_2 - s \sum_{i=1}^n 2(\mathbf{w}_2^\top \mathbf{x}_i - y_i) \mathbf{x}_i = \sum_{i=1}^n \alpha_i^2 \mathbf{x}_i - s \sum_{i=1}^n \gamma_i^2 \mathbf{x}_i = \sum_{i=1}^n \alpha_i^3 \mathbf{x}_i \quad (\text{with } \alpha_i^3 = \alpha_i^2 - s\gamma_i^2)$$

...

$$\mathbf{w}_t = \mathbf{w}_{t-1} - s \sum_{i=1}^n 2(\mathbf{w}_{t-1}^\top \mathbf{x}_i - y_i) \mathbf{x}_i = \sum_{i=1}^n \alpha_i^{t-1} \mathbf{x}_i - s \sum_{i=1}^n \gamma_i^{t-1} \mathbf{x}_i = \sum_{i=1}^n \alpha_i^t \mathbf{x}_i \quad (\text{with } \alpha_i^t = \alpha_i^{t-1} - s\gamma_i^{t-1})$$

形式上，以上论证是归纳法。 $\mathbf{w}$ 是训练向量 $\mathbf{w}_0$ (基本情况) 的线性组合。如果对 $\mathbf{w}_t$ 进行归纳假设，则对 $\mathbf{w}_{t+1}$ 也适用。

α_i^t 的更新规则如下：

$$\alpha_i^t = \alpha_i^{t-1} - s\gamma_i^{t-1}, \text{ 且: } \alpha_i^t = -s \sum_{r=0}^{t-1} \gamma_i^r.$$

由归纳过程，可得：

$$\mathbf{w}_t = \sum_{i=1}^n \alpha_i^t \mathbf{x}_i$$

即可以执行整个梯度下降的更新规则，而无需显式地表示出 $\mathbf{w}$ ，只需要记录这 n 个系数 $\alpha_1, \dots, \alpha_n$ 。

由于 $\mathbf{w}$ 可以写成训练集的线性组合，也可以将 $\mathbf{w}$ 与输入 $\mathbf{x}_i$ 的内积，用训练集输入之间的内积来表示，即：

$$\mathbf{w}^\top \mathbf{x}_i = \sum_{j=1}^n \alpha_j \mathbf{x}_j^\top \mathbf{x}_i.$$

由于：

$$\ell(\mathbf{w}) = \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2, \quad \mathbf{w}^\top \mathbf{x}_i = \sum_{j=1}^n \alpha_j \mathbf{x}_j^\top \mathbf{x}_i.$$

因此，重写的均方损失为：

$$\ell(\alpha) = \sum_{i=1}^n \left(\sum_{j=1}^n \alpha_j \mathbf{x}_j^\top \mathbf{x}_i - y_i \right)^2$$

在测试时，也只需要这些系数来对测试输入 $\mathbf{x}_t$ 进行预测，且可以根据测试点和训练点之间的内积来编写整个分类器：

$$h(\mathbf{x}_t) = \mathbf{w}^\top \mathbf{x}_t = \sum_{j=1}^n \alpha_j \mathbf{x}_j^\top \mathbf{x}_t.$$

- 测试时，对于测试输入 $\mathbf{x}_t$ ，有：

$$h(\mathbf{x}_t) = \mathbf{w}^\top \mathbf{x}_t = \sum_{j=1}^n \alpha_j \mathbf{x}_j^\top \mathbf{x}_t.$$

观察

为了学习具有均方损失的超平面分类器，所需要的唯一信息是所有数据向量对之间的内积。

回到上一个例子, $\phi(\mathbf{x}) = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_d \\ x_1 x_2 \\ \vdots \\ x_{d-1} x_d \\ \vdots \\ x_1 x_2 \cdots x_d \end{pmatrix}$.

内积 $\phi(\mathbf{x})^\top \phi(\mathbf{z})$ 可以表示为:

$$\phi(\mathbf{x})^\top \phi(\mathbf{z}) = 1 \cdot 1 + x_1 z_1 + x_2 z_2 + \cdots + x_1 x_2 z_1 z_2 + \cdots + x_1 \cdots x_d z_1 \cdots z_d = \prod_{k=1}^d (1 + x_k z_k).$$

2^d 项的和变成 d 项的乘积, 用上面的公式计算内积, 时间是 $O(d)$ 而不是 $O(2^d)$ 。

定义函数

$$\underbrace{k(\mathbf{x}_i, \mathbf{x}_j)}_{\text{称为 核函数 (kernel function)}} = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j).$$

对于 n 个样本的有限训练集，内积通常预先计算，并存储在一个核矩阵中：

$$K_{ij} = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j).$$

如果存储该矩阵 K ，则只需要在整个梯度下降算法中进行简单的内积查找和低维计算。

最终得到的分类器是：

$$h(\mathbf{x}_t) = \sum_{j=1}^n \alpha_j k(\mathbf{x}_j, \mathbf{x}_t).$$

从 $O(2^d)$ 到 $O(n^2)$

回顾:

$$\ell(\mathbf{w}) = \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2$$

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - s \left(\frac{\partial \ell}{\partial \mathbf{w}} \right) \quad \text{其中: } \frac{\partial \ell}{\partial \mathbf{w}} = \sum_{i=1}^n \underbrace{2(\mathbf{w}^\top \mathbf{x}_i - y_i)}_{\gamma_i : \mathbf{x}_i, y_i} \mathbf{x}_i = \sum_{i=1}^n \gamma_i \mathbf{x}_i$$

在新的 $\phi(\mathbf{x})$ 的高维空间中的训练上,
希望通过核来计算 γ_i , 而不需要计算任何 $\phi(\mathbf{x}_i)$ 甚至 $\mathbf{w}$ 。

由 $\mathbf{w} = \sum_{j=1}^n \alpha_j \phi(\mathbf{x}_j)$ 和 $\gamma_i = 2(\mathbf{w}^\top \phi(\mathbf{x}_i) - y_i)$, 可得出: $\gamma_i = 2(\sum_{j=1}^n \alpha_j K_{ij} - y_i)$ 。

$$\alpha_i^{t+1} \leftarrow \alpha_i^t - 2s \left(\sum_{j=1}^n \alpha_j^t K_{ij} - y_i \right)$$

由于有 n 个这样的更新要做 ($\alpha_i, i = 1, \dots, n$), 在转换后的空间中每次梯度更新的工作量为 $O(n^2)$ ——远远小于 $O(2^d)$ 。

1 特征扩展

- 手动特征扩展
- 手动特征扩展
- 手动特征扩展

2 核技巧

- 核技巧
- 均方损失梯度下降
- 内积计算

3 通用的核方法

- 常见的核函数
- 核函数

常见的核函数

下面是一些常见的核函数:

- **线性核函数:** $K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$
(线性核相当于只使用一个好的线性分类器——但是如果数据的维数 d 很高, 那么使用核矩阵会更快。)
- **多项式核函数:** $K(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}^\top \mathbf{z})^d$
- **径向基函数 (Radial Basis Function, RBF)(高斯核):** $K(\mathbf{x}, \mathbf{z}) = e^{\frac{-\|\mathbf{x}-\mathbf{z}\|^2}{\sigma^2}}$
RBF 核是最常用的核函数! 也是一个通用的逼近器! 其对应的特征向量是无限维的, 无法计算。然而, 存在非常有效的低维近似 (见[NeurIPS2008 参考论文](#))。
- **指数核函数:** $K(\mathbf{x}, \mathbf{z}) = e^{\frac{-\|\mathbf{x}-\mathbf{z}\|}{2\sigma^2}}$
- **拉普拉斯核函数:** $K(\mathbf{x}, \mathbf{z}) = e^{\frac{-|\mathbf{x}-\mathbf{z}|}{\sigma}}$
- **Sigmoid 核函数:** $K(\mathbf{x}, \mathbf{z}) = \tanh(\mathbf{a}\mathbf{x}^\top + c), \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

思考: 任何函数 $K(\cdot, \cdot) \rightarrow \mathcal{R}$ 都可以用来做核函数吗?

回答: 不是的, 矩阵 $K(\mathbf{x}_i, \mathbf{x}_j)$ 在经过一些变换 $\mathbf{x} \rightarrow \phi(\mathbf{x})$ 后, 必须对应于实际的內积。这种情况当且仅当 K 为半正定时才成立。

半正定矩阵的定义

矩阵 $A \in \mathbb{R}^{n \times n}$ 对 $\forall \mathbf{q} \in \mathbb{R}^n, \mathbf{q}^\top A \mathbf{q} \geq 0$ 是半正定的 (positive semi-definite, p.s.d.)。

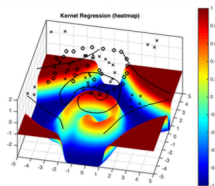
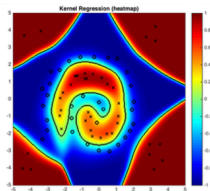
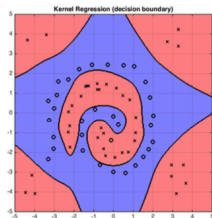
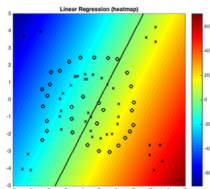
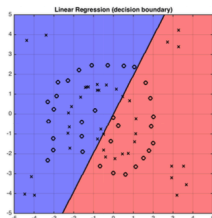
由于 $K_{ij} = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$, 可记为 $K = \Phi^\top \Phi$, 其中 $\Phi = [\phi(\mathbf{x}_1), \dots, \phi(\mathbf{x}_n)]$.

由此可知 K 是 p.s.d, 因为 $\mathbf{q}^\top K \mathbf{q} = (\Phi^\top \mathbf{q})^2 \geq 0$ 。

反过来, 如果任意矩阵 A 是 p.s.d, 则可以将其分解为 $A = \Phi^\top \Phi$, 以得到 Φ 的某种实现。

我们甚至可以在集合、字符串、图形和分子上定义核函数。

例子: RBF 核函数



该例子展示了核函数如何解决线性分类器无法解决的问题。在这种情况下，RBF 可以很好地处理决策边界。

The End